《现代C++编程实现》电子版：https://leanpub.com/c01

语句

教学资源：https://github.com/hwdong-net/cplusplus17

YouTube频道： hwdong

博客： hwdong-net.github.io

B站: hw-dong

简单语句、复合语句和控制语句

简单语句

- 最简单的语句是只有一个分号的**空语句**。

```
;
```

- 变量定义语句

```
int ival = 3, jval;
auto radius = 2.15;
```

- 表达式后面跟一个分号；构成了**表达式语句**。

```
std::cout<<ival;
```

复合语句

- 花括号{和}括起来的一系列语句构成一个**复合语句**（也称为**程序块**或**语句块**）。

```
#include <iostream>
int main()
{
    auto a=1,b=0;
    {
        a=3;
        auto b = 5;
    }
    std::cout<<a<<'\t'<<b<<std::end;
}
```

复合语句

- 内部块中的b是独立的局部变量，生命周期仅在块内，结束后被销毁。因此，输出的b是外部的b，值为0。

```
#include <iostream>
int main() {
    auto a = 1, b = 0;
    {
        a = 3;
        auto b = 5;       // 内部块中的局部变量b
    }
    std::cout << a << '\t' << b << std::endl; // 输出: 3 0
}
```

控制语句

- 条件语句

  满足某个条件，执行相应程序块

- 循环语句

  满足某个条件，重复执行相应程序块

- 跳转语句

  改变程序执行流程、从当前位置跳转到其他位置的控制语句。

条件语句

if 语句

switch语句

if语句

if 语句是最常用的条件语句，基本形式如下：

```
if (表达式)
    语句块
```

- **表达式**：一个能转换为 bool 的表达式（布尔表达式、非零值转为 true，零值转为 false）。
- 如果表达式为true，执行语句块；否则跳过。

示例

```
double score;
std::cin >> score;
if (score < 60)
    std::cout << "不及格！" << std::endl;
```

- 输入小于60时，输出“不及格！”；否则不输出。

if-else语句

当需要在条件为假时执行另一段代码时，可以使用**if-else**：

```
if (表达式)
    语句块1
else
    语句块2
```

```
double score;
std::cin >> score;
if (score < 60)
    std::cout << "不及格！" << std::endl;
else
    std::cout << "及格了！" << std::endl;
```

- 输入小于60时，输出“不及格！”；否则输出“及格了！”。

else if语句

当需要检查多个条件时，可以使用**else if**：

```
if (表达式1)
    语句块1
else if (表达式2)
    语句块2
else
    语句块3
```

示例

```
double score;
std::cin >> score;
if (score < 60)
    std::cout << "不及格！" << std::endl;
else if (score < 70)
    std::cout << "及格！" << std::endl;
else if (score < 80)
    std::cout << "中等！" << std::endl;
else if (score < 90)
    std::cout << "良好！" << std::endl;
else
    std::cout << "优秀！" << std::endl;
```

- 根据`score`的值，输出相应的评价。

注意if-else匹配

- **if**和**else**的匹配遵循“**就近原则**”：在没有明确花括号 **{}** 限制的情况下，**else子句**会与离它最近的未匹配的**if子句**配对。
- 例如：

```
double score;
std::cin >> score;
if (score >= 60)
    if (score > 90)
        std::cout << "优秀！" << std::endl;
else
    std::cout << "不及格！" << std::endl;
```

```
double score;
std::cin >> score;
if (score >= 60)
    if (score > 90)
        std::cout << "优秀！" << std::endl;
else
    std::cout << "不及格！" << std::endl;
```

- 这里的else 与内层的if (score > 90) 匹配，导致60到90之间的分数被误判为“不及格！”。

- 正确写法应使用花括号明确范围：

```
if (score >= 60) {
    if (score > 90)
        std::cout << "优秀！" << std::endl;
} else
    std::cout << "不及格！" << std::endl;
```

- 当一个if 或else 块里有多条语句时，也要用花括号{} 括起来，不然其含义就不对了。如：

```
#include <iostream>

int main() {
    auto a{0};
    std::cin >> a;
    if (a < 0)
        std::cout << "a=" << a << std::endl;
        std::cout << "这是一个负数" << std::endl;
    return 0;
}
```

```
8
这是一个负数
```

- 正确做法是用{}包围2条输出语句，构成if 子句的语句块。

```
if (a < 0) {
    std::cout << "a=" << a << std::endl;
    std::cout << "这是一个负数" << std::endl;
}
```

再如:

```
int a = 100;
if (a < 0)
    std::cout << "a的绝对值是";
    std::cout << -a;
else
    std::cout << "a的绝对值是";
    std::cout << a;
```

```
int a = 100;
if (a < 0) {
    std::cout << "a的绝对值是";
    std::cout << -a;
} else {
    std::cout << "a的绝对值是";
    std::cout << a;
}
```

switch语句

- 格式:

```
switch(可转化为整型的表达式){
    case (整型或枚举型)常量表达式 1:
        程序块 1
    case (整型或枚举型)常量表达式 2:
        程序块 2
    //...
    default:
        默认程序块
}
```

```
int x;
std::cin>>x;
switch(x){
    case 0:
    case 1:
        std::cout<<"x 是 0 或 1\n";
        break;    //break 关键字用于跳出 switch
    case 2:
        std::cout<<"x 是 2\n";
        break;    //break 关键字用于跳出 switch
    default:
        std::cout<<"x 不是 0，1， 2\n";
        break;    //break 关键字用于跳出 switch
}
```

```
unsigned vowelCnt = 0, nonVowelCnt=0;
char ch;
while(std::cin>>ch){
    switch(ch){
        case 'a':
        case 'e':
        case 'i':
        case 'o':
        case 'u':
            vowelCnt++;
            break;              //跳出 switch 循环
        default:
            nonVowelCnt++;
            break;              //跳出 switch 循环
    }
}
```

关于 switch 有几个语法点：

- case 标签必须是整型常量表达式
- case 后面的常量表达式值不能相同
- 某个 case 里定义变量，必须加花括号，否则假如执行其他 case 会出现“作用域里的该变量定义未初始化”
- default 标签可以省略
- break 关键字定义的 break 语句用于跳出整个 switch 语句。

```
//下面的switch 省略了default 标签
int  x,y;
switch(x){
      case  3.14:                          //错：case  标签必须是整型常量表达式
            //...  do  something
            break;
      case  y:                             //错：case  标签必须是整型常量表达式
            //...  do  something
            break;
}
```

```
int  x;
//...
//下面的有 2 个 case 的标签值都是 1，不能相同！
switch(x){
        case  1:
                /...
                break;
        case  1:
                //...
                break;
        case  2:
                //...
                break;
}
```

```
switch (v) {
case 1: int x = 0; //  初始化
        std::cout << x << '\n';
        break;
default:                    //  编译错误：因为default标签，可能导致 'x' 未初始化
       std::cout << "default\n";
       break;
}
```

```
switch (v) {
case 1: {    int x = 0;
        std::cout << x << '\n';
        break;
          }           //   'x' 的作用域在此结束
default: std::cout << "default\n"; //  无错误
        break;
}
```

if/switch语句中的初始化语句

if(初始化语句;表达式)

switch(初始化语句;表达式)

```
auto var = doSomething();
if (condition(var)) {
    // if块
} else {
    // else块
}
```

在C++17之前，若需在if 语句中使用变量，必须先在外部定义：

C++17允许将变量初始化嵌入if 语句：

```
if (auto var = doSomething(); condition(var)) {
    // if块
} else {
    // else块
}
```

```
auto var = doSomething();
if (condition(var)) {
    // if块
} else {
    // else块
}
```

在C++17之前，若需在if 语句中使用变量，必须先在外部定义：

C++17允许将变量初始化嵌入if 语句：

```
if (auto var = doSomething(); condition(var)) {
    // if块
} else {
    // else块
}
```

使代码更加简洁，另外，var只属于if语句，不会污染周围环境

```cpp
#include <iostream>
#include <string>
int main() {
    const std::string s = "Hello, my blog is: https://hwdong-net.github.io";
    const auto it = s.find("Hello");
    if (it != std::string::npos) {
        std::cout << it << " Hello\n";
    }

    const auto it2 = s.find("hwdong");
    if (it2 != std::string::npos) {
        std::cout << it2 << " hwdong\n";
    }
    return 0;
}
```

代码中用不同名字it 和 it2 区分2次查找的结果

```cpp
#include <iostream>
#include <string>
int main() {
    const std::string s = "Hello, my blog is: https://hwdong-net.github.io";
    if (const auto it = s.find("Hello"); it != std::string::npos) {
        std::cout << it << " Hello\n";
    }

    if (const auto it = s.find("hw-dong"); it != std::string::npos) {
        std::cout << it << " hwdong\n";
    }
    return 0;
}
```

两个if语句均使用相同的变量名it，避免了为每次查找起不同的名字，代码更简洁。
每个it的作用域仅限其所属的if语句，彼此独立，不会冲突。

示例：分数和等级

```
#include <iostream>
int main() {
    std::cout << "请输入分数（0-2）: ";
    switch (int score; std::cin >> score, score) {
        case 0:
            std::cout << "不及格\n";
            break;
        case 1:
            std::cout << "及格\n";
            break;
        case 2:
            std::cout << "优秀\n";
            break;
        default:
            std::cout << "分数无效\n";
    }
    return 0;
}
```

循环语句

循环语句用于重复执行代码块。

C++提供**while**、**do-while**、**for** 和**Range for** （基于范围的for）循环。

while语句

- while 循环在条件为真时重复执行代码块，基本形式如下：

```
while (表达式)
    语句块
```

  - 先判断表达式是否为 `true`（非零值）。
  - 如果为`true`，执行语句块；否则跳出循环。
  - 表达式每次循环开始前都会重新计算。

示例：计算阶乘

```
#include <iostream>
int main() {
    int n, i{1}, factorial{1};
    std::cout << "请输入一个正整数: ";
    std::cin >> n;
    while (i <= n) {
        factorial *= i;
        ++i;
    }
    std::cout << n << "的阶乘是: " << factorial << std::endl;
    return 0;
}
```

示例：计算平均分

下面程序从键盘输入一组成绩，最后输出平均分：

```
#include <iostream>

int main() {
    double score, average{0.0};
    int num{0};
    while (std::cin >> score) {
        if (score < 0) {
            break;  // 跳出 while 循环
        }
        average += score;
        ++num;
    }
    if (num > 0) {
        std::cout << "平均成绩是: " << average / num << std::endl;
    } else {
        std::cout << "没有输入任何有效成绩。\n";
    }
    return 0;
}
```

如果循环条件永远为 true， 会形成无限循环。例如：

```
while (true) { /* ... */ }  // 无限循环
```

break 语句

- 可以使用 break 关键字跳出 switch 语句。 同样， break 关键字也可以用于提前终止循环语句。

break：跳出循环

```
double score, average{ 0 };
auto num{ 0 };
while(std::cin >> score){
        if (score < 0)
                break;          //跳出while循环
         average += score;
        num++;
}
std::cout << "平均成绩是：" << average / num << std::endl;
```

do-while

- while语句的另外一个变种是所谓的**do-while**语句。

  do

  　程序块

  while(表达式);

dh1

do-while

```
double score, average{0};
    auto num{0};
    std::cin >> score;          //注意：先输入得到一个分数
    do {
            if (score < 0)
                    break;          //跳出while循环
            average += score;
            num++;
    } while (std::cin >> score);
    std::cout << "平均成绩是：" << average / num << std::endl;
```

不要忘记分号

幻灯片 42

dh1 dong hongwei, 2019/12/18

continue

- 关键字**continue**用于直接中断循环体里的后续语句执行，回到循环开头重新执行循环。

```
int main() {
    double score, average = {0};
    auto num{0};
    while (std::cin >> score) {
        if (score < 0)
            continue;   //回到循环开头环
        average += score;
        num++;
    }
    std::cout << "平均成绩是：" << average / num << std::endl;
}
```

for循环语句

for (初始表达式; 条件表达式; 后处理表达式)

程序块

```
//计算1到100整数之和的代码：

auto s{0} ;
for(auto i{1} ; i<=100; i++)
   s += i;
std::cout<<"1到100之间的整数之和是："<<s<<std::endl;
```

for循环语句

for (初始表达式；条件表达式；后处理表达式)

程序块

```
for (初始化表达式; 条件表达式; 后处理表达式)
    语句块
```

1. **初始化表达式：** 首先执行初始化表达式。 通常用于声明和初始化循环控制变量。 这个表达式只会在循环开始前执行一次。
2. **条件表达式：** 接下来，计算条件表达式 的值。
3. **循环执行：**

   如果条件表达式的值为 true （非零值），则执行循环体的程序块。

执行完程序块后，执行后处理表达式。 通常用于更新循环控制变量的值。

4. **重复判断：** 再次计算条件表达式的值，重复步骤 3，直到条件表达式 的值为 false（零值），此时循环结束。

示例：计算 1 到 100 整数之和

```
auto s{0}; // 初始化累加器

for (auto i{1}; i <= 100; i++) // 循环100次
  s += i;                      // 累加

std::cout << "1 到 100 之间的整数之和是: " << s << std::endl;
```

- `auto i{1}`：**初始化表达式**。在循环开始前执行一次，将循环控制变量 i 初始化为 1。
- `i <= 100`：**条件表达式**。在每次循环开始前判断。如果条件为真（i ≤ 100），则执行循环体；否则循环结束。
- `s += i`：**循环体**。执行循环的主要操作，将 i 的值累加到变量 s 中。
- `i++`：**后处理表达式**。在循环体执行完毕后执行，将 i 的值自增 1。
- 然后流程回到条件表达式 `i <= 100`，再次判断是否继续循环。如此反复，直到条件为假时退出。

for 循环与 while 循环的等价性

for (初始表达式; 条件表达式; 后处理表达式)

程序块

```
初始化表达式;
while( 条件表达式){
    程序块
    后处理表达式;
}
```

- 计算1到100的整数和可用`while`循环计算：

```
auto s{0};
auto i{1}; // 初始化表达式

while (i <= 100) { // 条件表达式
  s += i;
  i++; // 后处理表达式
}

std::cout << "1 到 100 之间的整数之和是: " << s << std::endl;
```

for 循环的灵活性

- 它的 "初始化表达式" 和 "后处理表达式" 都可以省略。

```
double score, average{ 0 };
auto num{ 0 };
for(; std::cin >> score; ){
    if (score < 0)
        break;  //跳出while循环
    average += score;
    num++;
}
```

for 循环的灵活性

- 也可以将部分逻辑移到条件表达式：

```
double score, average{0.0};
auto num{0};
for (; std::cin >> score && score >= 0;) {
  average += score;
  num++;
}
```

break 和 continue 语句

在 for 循环中的作用与在 while 循环中相同：

- break： 立即终止循环。
- continue： 跳过当前迭代的剩余部分，直接进入下一次迭代。

break 和 continue 语句

```
#include <iostream>

int main() {
  for (auto i{1}; i <= 100; i++) {
    if (i % 3 != 0)
      continue; // 停止后续语句执行，回到循环的条件表达式 "i <= 100"
    std::cout << i << std::endl;
  }
  return 0;
}
```

范围 for 循环 (Range-based for loop)

```
for (声明 : 范围)
    语句块
```

- **声明**：用于声明一个变量，该变量将依次存储 范围 中的每个元素。通常使用 auto 关键字来让编译器自动推断变量类型。
- **范围**：范围 可以是数组、std::vector、std::list 等容器，也可以是任何具有 begin() 和 end() 函数的对象（提供迭代器）。
- **语句块:** 循环体中的语句块代码会对当前声明的变量进行操作。

示例：遍历数组

```
#include <iostream>

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    for (int element : arr) {
        std::cout << element << ' ';
    }
    std::cout << std::endl;  // 输出：10 20 30 40 50
    return 0;
}
```

优点:

- **简洁易读:** 语法简单，更容易理解。
- **避免越界:** 不再需要手动管理索引，避免数组越界等错误。
- **通用性:** 适用于各种容器和序列。

小结

- while：适合条件未知的循环。
- do-while：至少执行一次循环体。
- for：适合已知次数或计数循环，三个部分灵活可省略。
- 范围 for（C++11）：最简洁的遍历方式，推荐用于容器和数组。
- break、continue 用于控制循环流程。

跳转语句

- break和continue
- Goto
- return

goto语句

goto 语句允许程序无条件跳转到同一函数内的指定标签处。：

```
goto 标签名;   // 跳转到标签

标签名:        // 标签定义（冒号结尾）
    语句;
```

```cpp
int main() {
    double score, average{0.0};
    int num{0};

    for (; std::cin >> score; ) {
        if (score < 0)
            goto label;  // 跳转到 label
        average += score;
        ++num;
    }
label:
    if (num > 0) {
        std::cout << "平均成绩是：" << average / num << std::endl;
    } else {
        std::cout << "没有输入任何有效成绩。\n";
    }
    return 0;
}
```

goto 的使用建议

- **强烈不推荐在现代 C++ 中使用 goto**，因为它会导致代码混乱、难以维护（即“意大利面条式代码”）。
- 常见替代方案：用 break 跳出循环、用 return 提前结束函数、用异常处理错误、用状态标志控制流程。
- **唯一可能的合理场景**：在嵌套多层循环中需一次性跳出多层时，goto 可能比多重 break 更简洁，但仍需谨慎。

总结

本章介绍了C++中的语句，特别是控制语句：

- **简单语句**：如变量定义、表达式语句。
- **复合语句**：用{}括起的语句块。
- 控制语句：
  - 条件语句：if、switch。
  - 循环语句：while、do-while、for、范围-based for。
  - 跳转语句：break、continue、return、goto。

控制台游戏-Pong游戏

- 参见书

关注我

博客： hwdong-net.github.io

Youtube频道

hwdong

@hwdong · 5.01K subscribers · 558 videos

博客：https://hwdong-net.github.io >

youtube.com/c/4kRealSound and 4 more links